

使用 Java 建構 gRPC 服務

來源：<https://codelabs.developers.google.com/codelabs/cloud-grpc-java?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將瞭解如何建構以 Java 為基礎的服務，透過 gRPC 公開 API。然後為 gRPC 服務編寫 Java 指令列用戶端。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [建立 gRPC 服務](#)
4. [使用服務](#)
5. [影視串流服務](#)
6. [恭喜！](#)

步驟 1 · 約 1 分鐘

1. 總覽

gRPC 是 Google 開發的語言和平台中立遠端程序呼叫 (RPC) 架構和工具組。您可以使用通訊協定緩衝區定義服務，這是一套功能強大的二進位序列化工具集和語言。然後，您就可以從各種語言的服務定義中，產生慣用的用戶端和伺服器存根。

在本程式碼研究室中，您將瞭解如何建構以 Java 為基礎的服務，透過 gRPC 架構公開 API，然後編寫用戶端來使用產生的 gRPC 用戶端側存根。

課程內容

- 通訊協定緩衝區語言
 - 如何使用 Java 實作 gRPC 服務
 - 如何使用 Java 實作 gRPC 用戶端
-

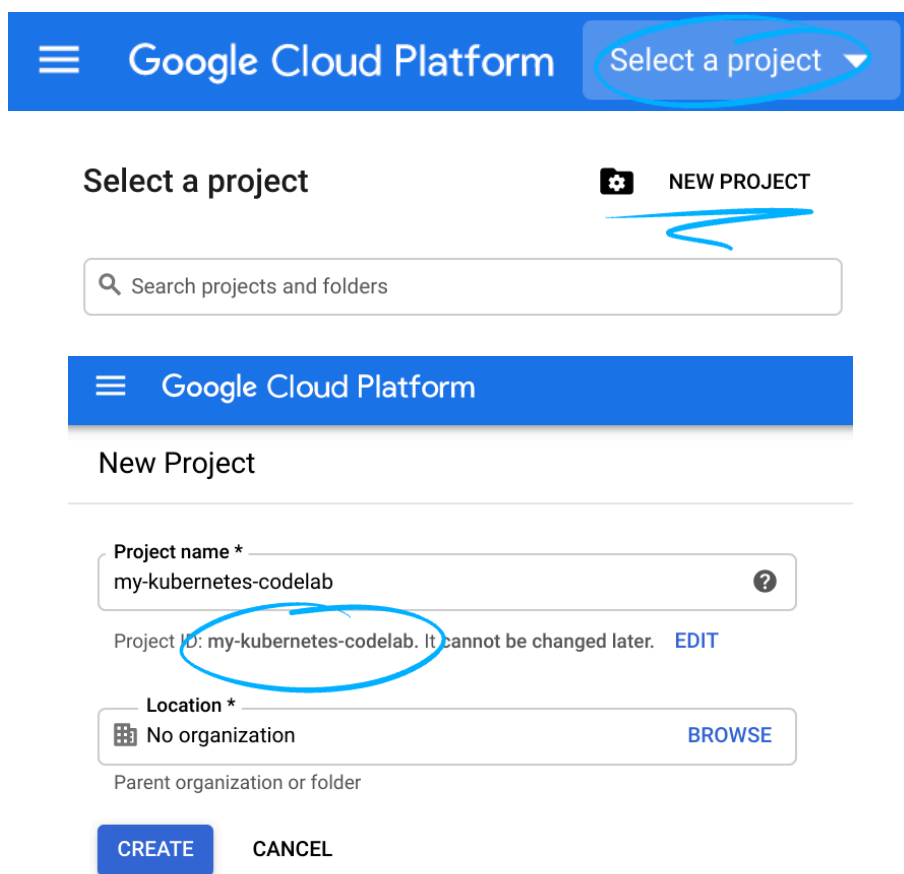
步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：`console.cloud.google.com`。



The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu. Below this, there's a 'Select a project' section with a search bar and a 'NEW PROJECT' button. The 'NEW PROJECT' button is circled in blue. Below the search bar, there's a 'New Project' form. The form has three main sections: 'Project name', 'Project ID', and 'Location'. The 'Project name' field is filled with 'my-kubernetes-codelab'. The 'Project ID' field is filled with 'my-kubernetes-codelab' and is circled in blue. The 'Location' field is set to 'No organization'. At the bottom of the form, there are 'CREATE' and 'CANCEL' buttons.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

Google Cloud Shell

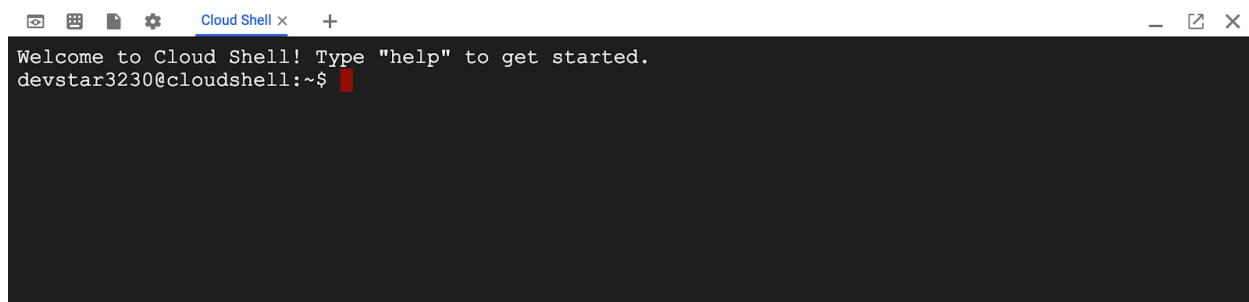
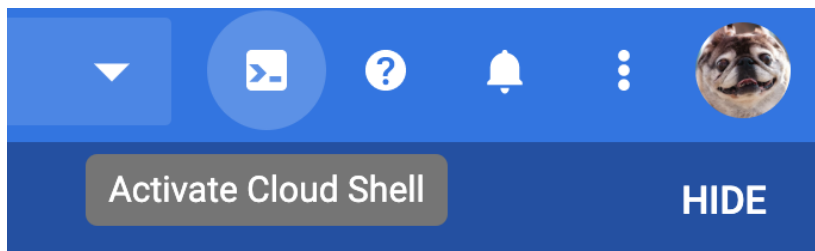
雖然您可以在電腦上操作這個程式碼研究室，但我們將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

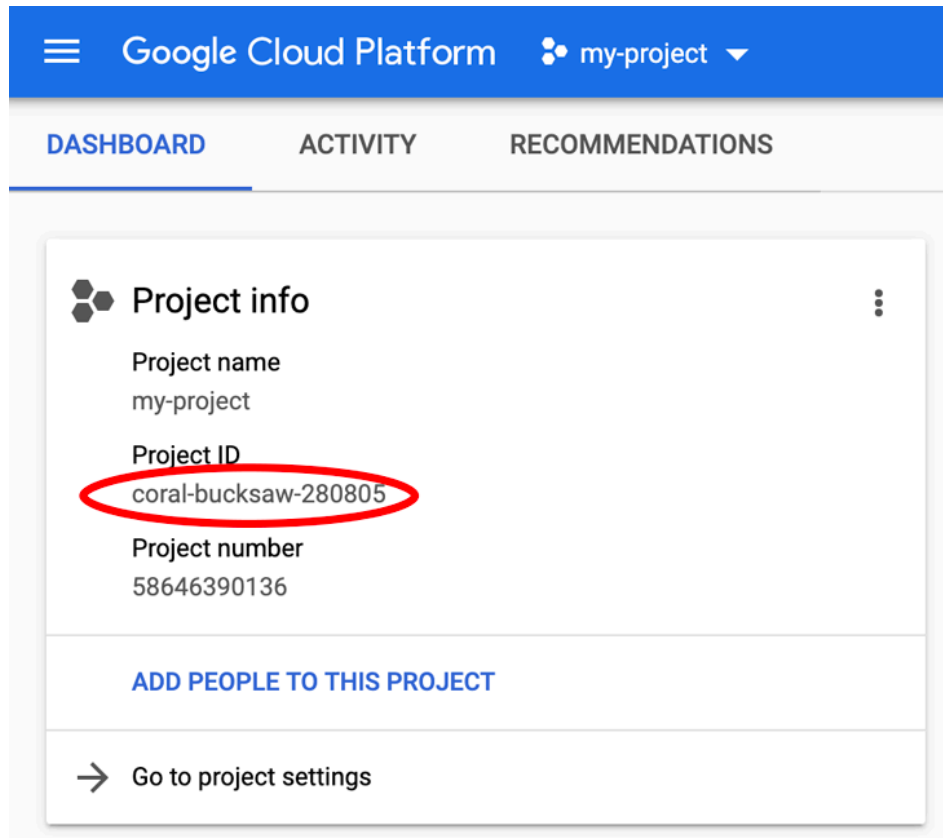
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

步驟 3 · 約 5 分鐘

3. 建立 gRPC 服務

使用 Maven 建立新的 Java 專案：

```
$ mvn archetype:generate -DgroupId=com.example.grpc \
-DartifactId=grpc-hello-server \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
$ cd grpc-hello-server
```

新增 gRPC 定義檔

在 gRPC 中，服務酬載 (要求和回應) 和服務作業必須以 IDL (介面定義語言) 擷取。gRPC 使用 Protobuffer 3 語法定義訊息酬載和作業。讓我們為簡單的 Greeting Service 建立 proto 檔案，其中包含 Hello Request 和 Hello Response。

首先，建立新的 proto 目錄來存放新的 proto 檔案：

```
$ mkdir -p src/main/proto
```

接著，建立新的 proto 檔案 `src/main/proto/GreetingService.proto`。

你可以使用 `vim`, `nano`, 或 `emacs` 編輯檔案：

src/main/proto/GreetingService.proto

```

syntax = "proto3";
package com.example.grpc;

// Request payload
message HelloRequest {
    // Each message attribute is strongly typed.
    // You also must assign a "tag" number.
    // Each tag number is unique within the message.
    string name = 1;

    // This defines a strongly typed list of String
    repeated string hobbies = 2;

    // There are many more basics types, like Enum, Map
    // See https://developers.google.com/protocol-buffers/docs/proto3
    // for more information.
}

message HelloResponse {
    string greeting = 1;
}

// Defining a Service, a Service can have multiple RPC operations
service GreetingService {
    // Define a RPC operation
    rpc greeting(HelloRequest) returns (HelloResponse);
}

```

新增 gRPC 依附元件和外掛程式

定義完成後，我們就能從這個檔案產生伺服器端和用戶端存根。您需要新增 gRPC 依附元件和外掛程式。

首先，請將 gRPC 依附元件新增至 `pom.xml`：

pom.xml

```

<project>
  ...
  <dependencies>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-netty-shaded</artifactId>
      <version>1.24.0</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-protobuf</artifactId>
      <version>1.24.0</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-stub</artifactId>
      <version>1.24.0</version>
    </dependency>
    <dependency>
      <groupId>javax.annotation</groupId>
      <artifactId>javax.annotation-api</artifactId>
      <version>1.3.2</version>
    </dependency>
    ...
  </dependencies>
  ...
</project>

```

然後新增外掛程式：

pom.xml

```

<project>
  ...
  <dependencies>
    ...
  </dependencies>
  <build>
    <extensions>
      <extension>
        <groupId>kr.motd.maven</groupId>
        <artifactId>os-maven-plugin</artifactId>
        <version>1.6.2</version>
      </extension>
    </extensions>
    <plugins>
      <plugin>
        <groupId>org.xolstice.maven.plugins</groupId>
        <artifactId>protobuf-maven-plugin</artifactId>
        <version>0.6.1</version>
        <configuration>

<protocArtifact>com.google.protobuf:protoc:3.9.0:exe:${os.detected.classifier}
</protocArtifact>
        <pluginId>grpc-java</pluginId>
        <pluginArtifact>io.grpc:protoc-gen-grpc-
java:1.24.0:exe:${os.detected.classifier}</pluginArtifact>
        </configuration>
        <executions>
          <execution>
            <goals>
              <goal>compile</goal>
              <goal>compile-custom</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>

</project>

```

這個外掛程式會先偵測您用來建構應用程式的作業系統和硬體架構，然後使用這項資訊下載正確的平台專屬二進位檔，將 proto 檔案轉換為 Java 程式碼。

生成存根

建構應用程式時，外掛程式會將 proto 定義轉換為 Java 程式碼。

```
$ mvn -DskipTests package
```

如要查看產生的檔案，請按照下列步驟操作：

```
$ find target/generated-sources
```

導入服務

首先，請建立新的 `GreetingServiceImpl` 類別，實作 `greeting` 作業：

src/main/java/com/example/grpc/GreetingServiceImpl.java

```
package com.example.grpc;

import io.grpc.stub.StreamObserver;

public class GreetingServiceImpl extends GreetingServiceGrpc.GreetingServiceImplBase
{
    @Override
    public void greeting(GreetingServiceOuterClass.HelloRequest request,
        StreamObserver<GreetingServiceOuterClass.HelloResponse> responseObserver) {
        // HelloRequest has toString auto-generated.
        System.out.println(request);

        // You must use a builder to construct a new Protobuffer object
        GreetingServiceOuterClass.HelloResponse response =
        GreetingServiceOuterClass.HelloResponse.newBuilder()
            .setGreeting("Hello there, " + request.getName())
            .build();

        // Use responseObserver to send a single response back
        responseObserver.onNext(response);

        // When you are done, you must call onCompleted.
        responseObserver.onCompleted();
    }
}
```

gRPC 產生的伺服器端虛設常式一律為非同步實作。因此這個方法沒有傳回值。您必須改用 `StreamObserver` 回呼介面傳回資料。由於這是完全非同步的作業，您也需要呼叫 `onCompleted`，指出伺服器已完成傳送回應。

導入伺服器

最後，您需要啟動伺服器，監聽某個連接埠並註冊這項服務實作。編輯 `App` 類別及其主要方法：

src/main/java/com/example/grpc/App.java

```
package com.example.grpc;

import io.grpc.*;

public class App
{
    public static void main( String[] args ) throws Exception
    {
        // Create a new server to listen on port 8080
        Server server = ServerBuilder.forPort(8080)
            .addService(new GreetingServiceImpl())
            .build();

        // Start the server
        server.start();

        // Server threads are running in the background.
        System.out.println("Server started");
        // Don't exit the main thread. Wait until server is terminated.
        server.awaitTermination();
    }
}
```

最後，執行伺服器：

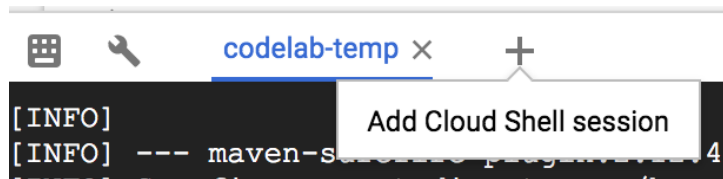
```
$ mvn -DskipTests package exec:java -Dexec.mainClass=com.example.grpc.App
...
Server Started
```

步驟 4 · 約 5 分鐘

4. 使用服務

產生器已產生所有用戶端存根。為簡化實驗室，我們將使用相同的 Maven 專案，但會新增具有新 main 方法的 `Client` 類別。

首先，點選「+」開啟新的 Cloud Shell 工作階段，這樣就不必終止伺服器：



在新工作階段中，切換至 `grpc-hello-server` 目錄：

```
$ cd grpc-hello-server
```

然後新增 `Client` 類別：

src/main/java/com/example/grpc/Client.java

```

package com.example.grpc;

import io.grpc.*;

public class Client
{
    public static void main( String[] args ) throws Exception
    {
        // Channel is the abstraction to connect to a service endpoint
        // Let's use plaintext communication because we don't have certs
        final ManagedChannel channel =
ManagedChannelBuilder.forTarget("localhost:8080")
        .usePlaintext(true)
        .build();

        // It is up to the client to determine whether to block the call
        // Here we create a blocking stub, but an async stub,
        // or an async stub with Future are always possible.
        GreetingServiceGrpc.GreetingServiceBlockingStub stub =
GreetingServiceGrpc.newBlockingStub(channel);
        GreetingServiceOuterClass.HelloRequest request =
        GreetingServiceOuterClass.HelloRequest.newBuilder()
        .setName("Ray")
        .build();

        // Finally, make the call using the stub
        GreetingServiceOuterClass.HelloResponse response =
        stub.greeting(request);

        System.out.println(response);

        // A Channel should be shutdown before stopping the process.
        channel.shutdownNow();
    }
}

```

最後，執行用戶端：

```

$ mvn -DskipTests package exec:java -Dexec.mainClass=com.example.grpc.Client
...
greeting: "Hello there, Ray"

```

大功告成！很簡單吧？

5. 影視串流服務

你還可以嘗試許多其他功能。舉例來說，您只要在 proto 檔案中，將 `stream` 關鍵字新增至要求或回應參數，即可輕鬆建構串流服務，例如：

src/main/proto/GreetingService.proto

```
syntax = "proto3";
package com.example.grpc;

...

// Defining a Service, a Service can have multiple RPC operations
service GreetingService {
    // MODIFY HERE: Update the return to streaming return.
    rpc greeting(HelloRequest) returns (stream HelloResponse);
}
```

更新伺服器，傳送多個回應而非單一回應。您可以透過多次呼叫 `responseObserver.onNext(...)` 來完成這項操作：

src/main/java/com/example/grpc/GreetingServiceImpl.java

```
package com.example.grpc;

import io.grpc.stub.StreamObserver;

public class GreetingServiceImpl extends GreetingServiceGrpc.GreetingServiceImplBase
{
    @Override
    public void greeting(GreetingServiceOuterClass.HelloRequest request,
        StreamObserver<GreetingServiceOuterClass.HelloResponse> responseObserver) {

        ...

        // Feel free to construct different responses if you'd like.
        responseObserver.onNext(response);
        responseObserver.onNext(response);
        responseObserver.onNext(response);

        // When you are done, you must call onCompleted.
        responseObserver.onCompleted();
    }
}
```

用戶端必須使用非同步存根，而非封鎖存根。更新用戶端程式碼，確保將 `stub` 型別更新為 `GreetingServiceStub`：

src/main/java/com/example/grpc/Client.java

```

package com.example.grpc;

import io.grpc.*;

// New import
import io.grpc.stub.*;

public class Client
{
    public static void main( String[] args ) throws Exception
    {
        final ManagedChannel channel =
ManagedChannelBuilder.forTarget("localhost:8080")
            .usePlaintext(true)
            .build();

        // Replace the previous synchronous code with asynchronous code.
        // This time use an async stub:
        GreetingServiceGrpc.GreetingServiceStub stub =
GreetingServiceGrpc.newStub(channel);

        // Construct a request
        GreetingServiceOuterClass.HelloRequest request =
            GreetingServiceOuterClass.HelloRequest.newBuilder()
                .setName("Ray")
                .build();

        // Make an Asynchronous call. Listen to responses w/ StreamObserver
        stub.greeting(request, new
StreamObserver<GreetingServiceOuterClass.HelloResponse>() {
            public void onNext(GreetingServiceOuterClass.HelloResponse response) {
                System.out.println(response);
            }
            public void onError(Throwable t) {
            }
            public void onCompleted() {
                // Typically you'll shutdown the channel somewhere else.
                // But for the purpose of the lab, we are only making a single
                // request. We'll shutdown as soon as this request is done.
                channel.shutdownNow();
            }
        });
    }
}

```

重新建構應用程式：

```
$ mvn -DskipTests package
```

在各自的 Cloud Shell 工作階段中，重新啟動伺服器 and 用戶端。

如何啟動伺服器：

```
$ mvn exec:java -Dexec.mainClass=com.example.grpc.App
...
Server Started
```

啟動用戶端：

```
$ mvn exec:java -Dexec.mainClass=com.example.grpc.Client
...
greeting: "Hello there, Ray"
greeting: "Hello there, Ray"
greeting: "Hello there, Ray"
```

步驟 6 · 約 1 分鐘

6. 恭喜！

涵蓋內容：

- 通訊協定緩衝區語言
- 如何使用 Java 實作 gRPC 伺服器
- 如何使用 Java 實作 gRPC 用戶端

後續步驟：

- 進一步瞭解 [gRPC Java](#)
- 在 GitHub 上查看更多 [gRPC Java 範例](#)
- 瞭解 [gRPC 中的串流](#)
- 進一步瞭解 [gRPC 到 REST 的轉碼](#)

提供意見

- 請花點時間填寫這份簡短的[問卷調查](#)
-